

Cooper Hoy | Cyber Security Engineering

If you had told me a few years ago that I would end up studying cyber security, I would not have believed you. I started college at Hawkeye Community College and transferred to Iowa State in 2023 as a software engineering major. By fall 2024, I switched to cyber security engineering.

At the time, changing majors made me feel like I was getting behind. Looking back, I don't really see it that way anymore. I had to spend some time figuring out what I enjoyed and what kind of work I could see myself doing for the long term. That process ended up being useful because it taught me that figuring something out sometimes means going down the wrong path first.

One thing that surprised me at Iowa State was how often my classes made me think about the people affected by technology, rather than just the technology itself. In Legal, Professional, and Ethical Issues in Cyber Security, I started thinking more seriously about what happens when security fails and who deals with the consequences. Most of the time, it is not the engineer or the company making the decision. It is usually a normal person using the system.

I also took Russia Today, which honestly seemed like a pretty random class when I signed up for it. It ended up helping me understand more about how technology, politics, and international conflicts overlap. That became much more real during my internships when I could see malicious traffic and login attempts coming from systems around the world.

Other classes changed how I look at projects in diverse ways. Microeconomics and Personal Finance made me think more about cost and decision-making. Software Project Management made it obvious that a technically great project can still fail if the team cannot manage its budget, schedule, or people.

Senior design has brought most of these ideas together. My team is building a solar-powered sensor network to detect wildfires earlier. It is one of the first school projects where the consequences feel bigger than getting a grade. If a system like ours were deployed, people could depend on it during an emergency. That changes the way you think about reliability and design decisions.

My technical classes also taught me that rushing usually creates more work later. In Computer Organization, I designed a five-stage RISC-V processor. I learned quickly that skipping planning because I wanted to start building only meant spending more time debugging later.

Operating Systems was another class that changed the way I think. Once I understood more about processes, memory, concurrency, and what was happening underneath the software, troubleshooting started to make more sense. Problems that used to feel random became easier to break apart.

Software Development Practices were useful for assorted reasons. I worked with the same team throughout the project, and it became obvious that writing clever code does not matter much if nobody else understands it. Good documentation, communication, and readable code became just as important as getting something to work.

Some of my favorite schoolwork has come from security courses where I built networks, configured servers, secured them, and then tried to attack what I had just built. It was fun, but it also showed me how small mistakes can become serious problems. One bad configuration, weak password, or forgotten service can completely change how secure a system is.

The experiences that tested me the most happened outside of the classroom.

During my internship at RSM, I noticed someone attempting thousands of password combinations against a client's VPN. My first reaction was blocking everything associated with traffic. The problem was that doing that could also block legitimate employees who needed the VPN to work.

Instead, we worked on an automated response that could stop the malicious activity without preventing normal users from connecting. That was one of the first moments where I felt like something I was doing in cyber security had an immediate effect on a real organization.

Before RSM, I worked at Cambridge Investment Research, where one of the things I helped with was building backup systems so the company could continue operating if something failed. Today, I worked at QCI, where I helped configure servers, networks, and cloud systems for clients while finishing school.

The situations are different, but the biggest lesson from all three jobs has been similar: do not jump straight to fixing something before you understand what is wrong. That sounds obvious, but it is surprisingly easy to forget when something breaks, and people want it to be fixed immediately.

A lot of what I know also did not come directly from a class.

At work, I have spent a lot of time reading vendor documentation, searching through forums, and asking engineers questions. Sometimes the answer is buried in a document nobody wants to read, and sometimes somebody with more experience can explain it in thirty seconds.

At home, I run my own small server setup. It breaks more often than I would like to admit, which has taught me more than if it worked perfectly all the time. I have spent plenty of nights reading documentation, digging through logs, changing configurations, breaking something else, and eventually getting everything running again.

I also build projects mainly because I want to learn how something works. One of them, Knext, scans networks and identifies known security weaknesses. Another, NOCstock, helps track network equipment and inventory. Nobody assigned those projects to me. I built them because I was curious, and over time I realized that this is the way I learn best. I like having an actual problem in front of me and figuring it out as I go.

Instructing other people has also been a bigger part of my college experience than I expected.

At the Iowa State IT Solution Center, I started as a support technician and eventually became a Team Lead. In that role, I helped lead fifteen student employees. One issue I noticed was that new employees were taking several weeks before they felt comfortable handling problems on their own.

I worked on developing a more organized training process that helped new employees become comfortable in two weeks instead of six. Building that process taught me a lot about communication. Knowing the answer yourself is one thing. Explaining it in a way that makes sense to somebody seeing it for the first time is completely different.

Since 2023, I have also mentored more than thirty students involved in FIRST Tech Challenge robotics teams in Grundy Center. Sometimes that means helping someone write their first program. Other times, it means helping troubleshoot a robot five minutes before it needs to run.

Those experiences have taught me something I try to remember now: if I cannot explain an idea simply, I do not understand it as well as I do.

My grades are also part of my experience at Iowa State, including the ones I am not especially proud of. Some of my earlier semesters were rough. I was working, taking difficult classes, and honestly was not always studying in a way that worked for me.

Eventually, I had to change how I managed my time and how I approached school. Spring 2026 ended up being one of my strongest semesters, including As in Operating Systems and Software Development Practices.

I would rather be honest about that than pretend everything went perfectly. My transcript shows some mistakes, but it also shows improvement. Learning how to recover from a bad semester was just as important as doing well in a good one.

I have tried to carry that same mindset into my career. I earned my Cisco Certified Support Technician certification, I am working toward my CCNA, and I continue building projects outside of school because there is always something else I realize I do not know yet.

Eventually, I would like to work as a sales engineer. What interests me about that role is that it requires both technical knowledge and the ability to communicate that knowledge to people who may not be engineers. My experiences in IT support, mentoring, consulting, and school have made me realize that I enjoy both sides of that.

When I think about what Iowa State has given me, I do not really think of a specific class or project first. The biggest thing I have gained is confidence that I can figure things out.

Technology changes too quickly for everything I learned today to stay relevant forever. A few years from now, I will be working with technologies that I have never used before, solving problems that are not covered in any of my current textbooks.

I am okay with that now.

I have learned how to admit when I do not know something, find the information or people that can help, test different ideas, make mistakes, and keep working until the problem starts to make sense.

That is the most important thing I will take with me from Iowa State.